

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA1522

Register Number	Shaik Ashraf
Name	192324088

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I Shaik Ashraf / 192324088 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:  _____

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are	Most constraints identified	Some constraints identified	Constraints poorly understood

		identified and prioritized accurately			
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

Question 1 – HDFS-Based Storage Solution for a Media Platform

Objective

Design an HDFS storage architecture that maintains availability above 99.9%, uses a replication factor of at most 3, and keeps storage growth cost efficient.

Key Constraints

- Availability must remain above 99.9%.
- Replication factor cannot exceed 3.
- Storage must scale as media data grows without unnecessary infrastructure cost.
- The design should tolerate node failures without losing data availability.

Proposed Architecture

- Use a multi-node HDFS cluster with a highly available NameNode arrangement.
- Use one active NameNode and one standby NameNode with shared or replicated metadata storage.
- Deploy multiple DataNodes across failure domains such as racks or physical zones.
- Set HDFS replication factor to 3 for important media objects, while avoiding unnecessary replicas beyond the constraint.
- Apply rack awareness so replicas are distributed across different racks.
- Use compression and suitable file formats such as Parquet/ORC for derived analytics data to reduce storage footprint.
- Use storage tiering or lifecycle policies to move older, less frequently accessed media to lower-cost storage where the platform permits it.

Replication Strategy

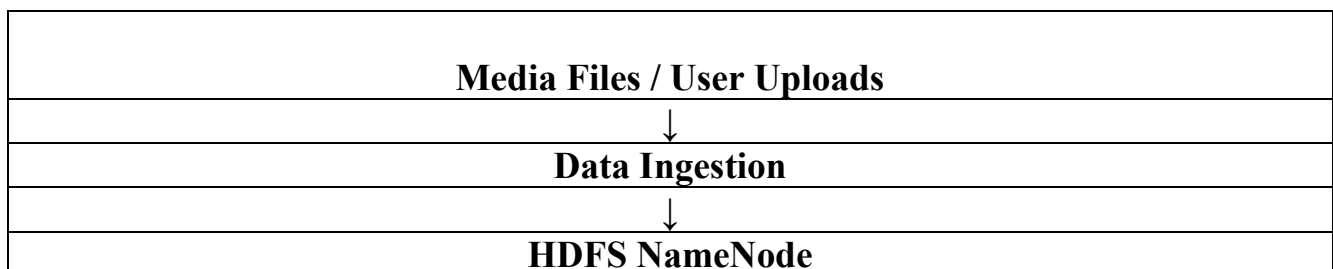
Component	Design Choice	Reason
NameNode	Active + Standby	Reduces single point of failure and supports high availability.
DataNodes	Multiple nodes across racks	Allows continued service after individual node/rack failures.
Replication	Factor = 3	Provides fault tolerance while respecting the maximum replication constraint.
Rack Awareness	Enabled	Prevents all copies from being placed in the same failure domain.
Growth	Horizontal scaling	Adds capacity by adding DataNodes rather than replacing the whole cluster.

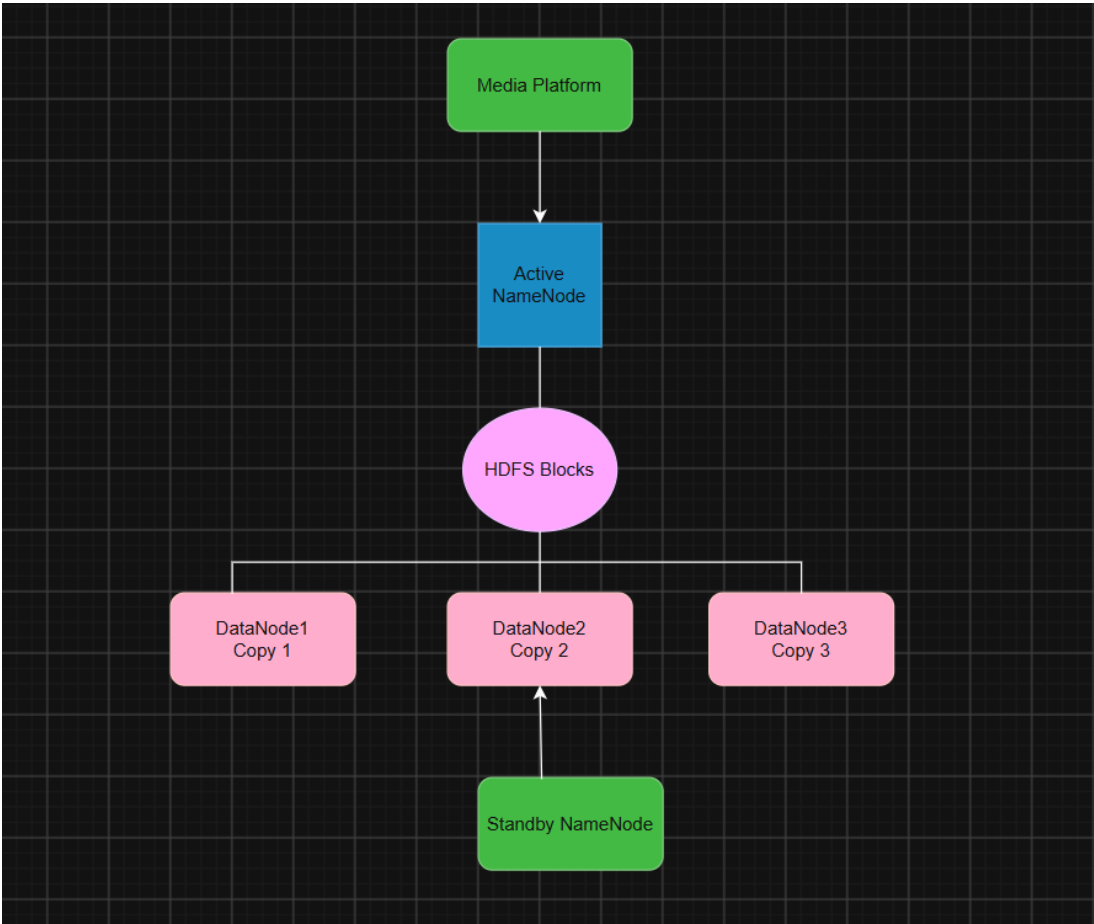
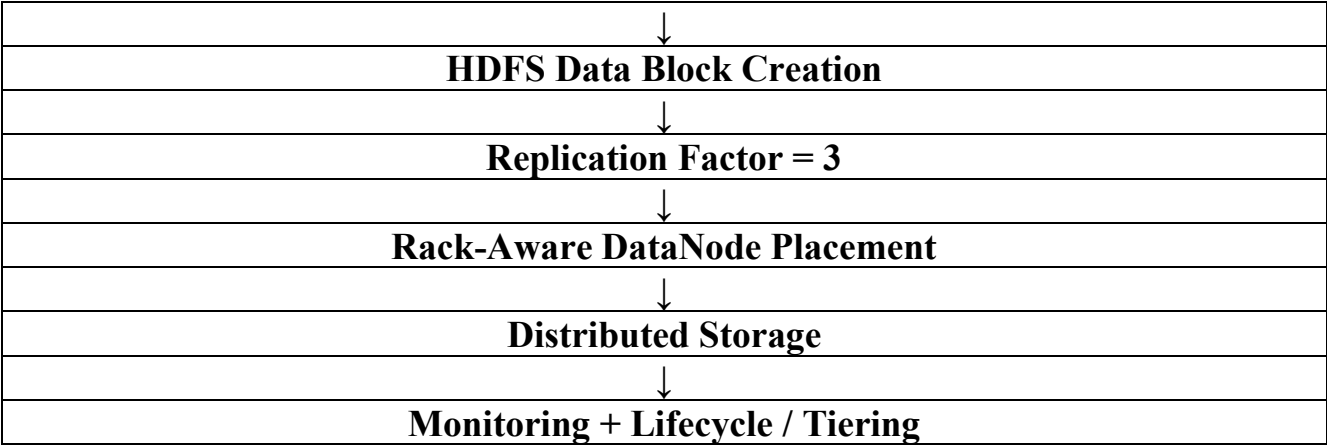
Why This Design Satisfies the Constraints

- A standby NameNode and distributed DataNodes reduce the impact of hardware failure.
- Replication factor 3 provides three copies while remaining within the stated limit.
- Rack-aware placement improves resilience against rack-level failures.
- Horizontal scaling makes storage expansion incremental and cost efficient.
- Compression and lifecycle-based storage management reduce unnecessary storage consumption.

Recommended Data Flow

Media Ingestion → HDFS Write → Block Distribution → Replication Across DataNodes/Racks → Metadata Management → Analytics/Processing → Lifecycle/Tiering





Question 2 – MapReduce Workflow for Log Processing Within 20 Minutes

Objective

Process large volumes of log records in a batch window of 20 minutes while operating with a limited number of cluster nodes.

Key Constraints

- Maximum batch processing window: 20 minutes.
- Cluster nodes are limited.
- The workload must be parallelized efficiently.
- Network and disk overhead should be minimized.

Proposed Job Design

- Store raw log files in HDFS and partition them into sufficiently large input blocks.
- Use multiple mapper tasks so independent input splits can be processed in parallel.
- Perform filtering and field extraction in the Mapper phase to reduce unnecessary data transferred to Reducers.
- Use a Combiner for associative and commutative aggregation operations such as counts and sums.
- Use a suitable partitioner to distribute keys evenly among reducers and prevent reducer hotspots.
- Use a limited but sufficient number of reducers based on the available nodes and data volume.
- Monitor task execution and tune input split size, mapper/reducer counts, and resource allocation to keep total execution below 20 minutes.

Task Distribution

Stage	Main Work	Distribution Strategy
Input	Read log blocks from HDFS	Input splits mapped across available nodes.
Mapper	Parse, filter, and emit key-value pairs	Run mapper tasks in parallel near the data.

Combiner	Perform local aggregation	Reduce intermediate data before shuffle.
Shuffle/Sort	Group records by key	Partition keys across reducers.
Reducer	Aggregate final results	Use parallel reducers within node/resource limits.
Output	Write processed results	Store final output in HDFS.



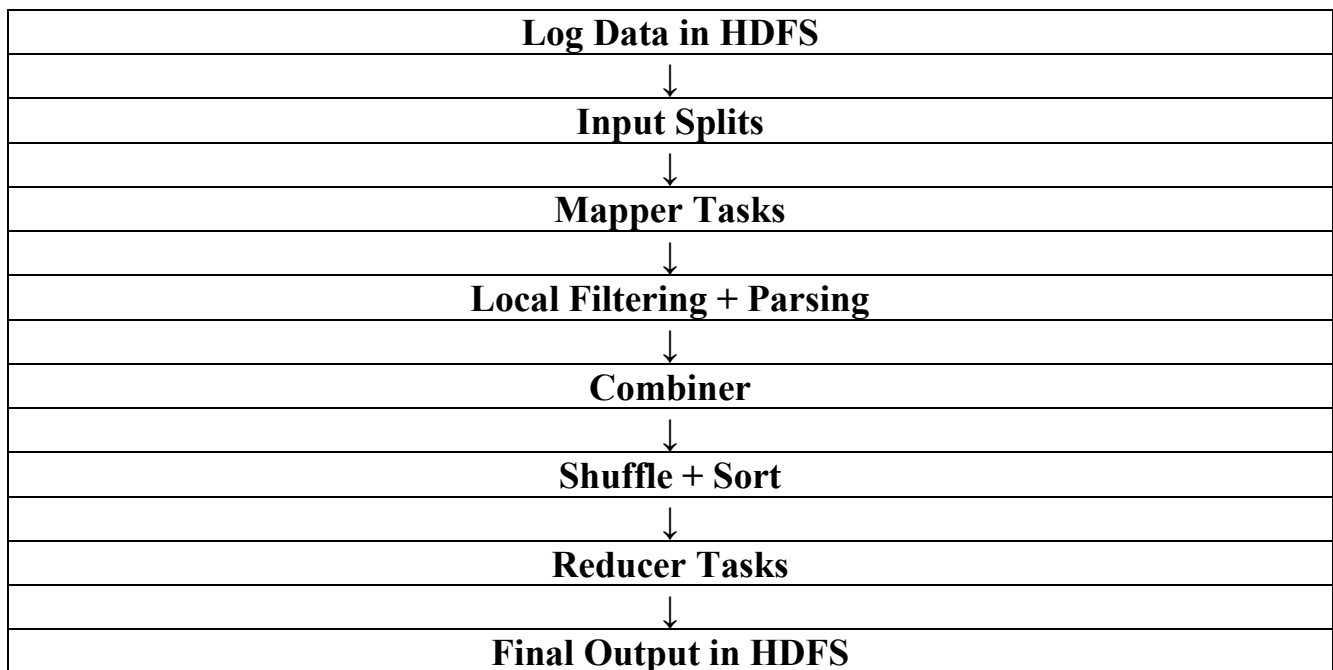
Performance Optimization Points

- Move computation close to data through HDFS data locality.
- Filter invalid or irrelevant records in the Mapper.
- Use a Combiner wherever mathematically valid.
- Avoid excessive small files because they create many mapper tasks and metadata overhead.

- Use balanced partitioning to avoid reducer skew.
- Tune memory, CPU, and container resources through YARN.

20-Minute Execution Strategy

- Target the map phase to finish substantially before the deadline so that shuffle and reduce phases have sufficient time.
- Run independent mapper tasks concurrently across all usable nodes.
- Reduce intermediate data before network transfer.
- Keep reducer workload balanced and avoid a single slow reducer becoming the bottleneck.



Question 3 – YARN Scheduler Policy for ETL, Reporting, and Machine Learning

Objective

Design a YARN scheduling policy that provides fairness, gives ETL jobs appropriate priority, and recovers effectively when nodes fail.

Key Constraints

- Fairness among workload classes must be maintained.
- ETL jobs require priority because they can feed downstream reporting and ML workloads.
- Node failures must not permanently stop applications.
- The policy must efficiently share limited cluster resources.

Recommended Scheduler Policy

Workload	Priority	Scheduling Approach	Reason
ETL	High	Guaranteed minimum capacity + priority	Feeds downstream workloads and is often time-sensitive.
Reporting	Medium	Fair-share capacity	Provides responsive access without starving ETL.
Machine Learning	Medium/Low	Fair-share with controlled concurrency	ML jobs can be resource intensive and long running.

YARN Queue Structure

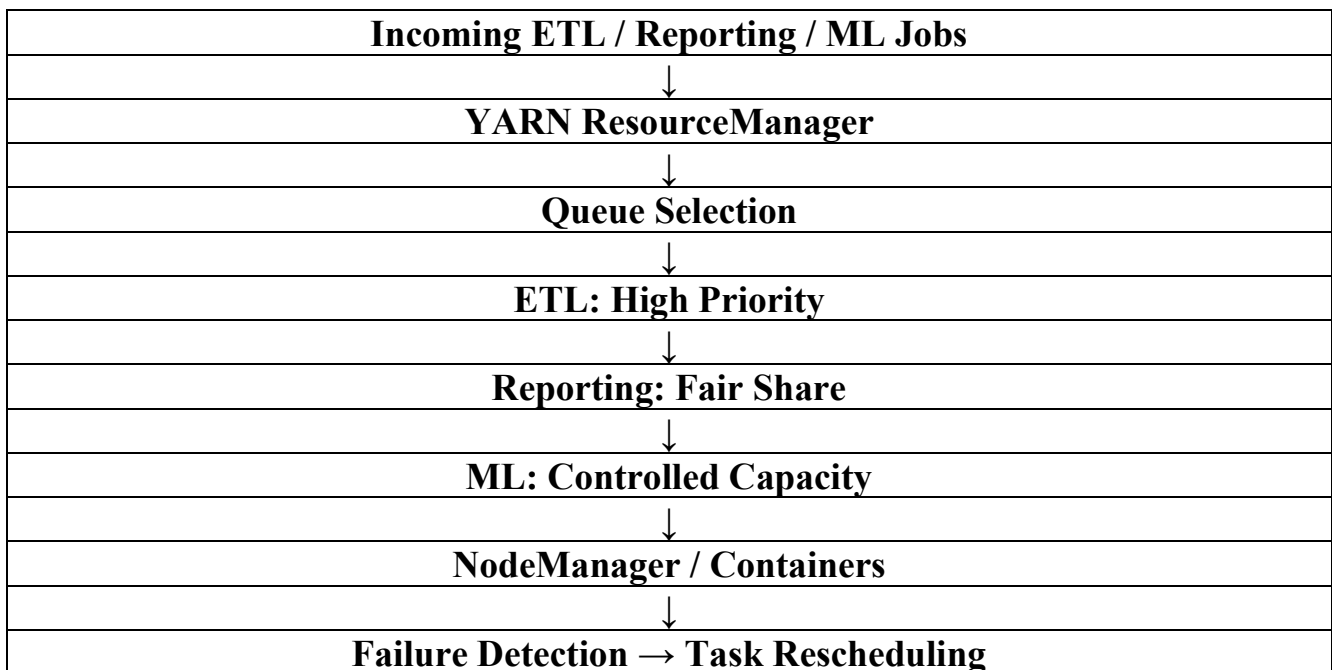
- Root Queue
- ETL Queue – reserved minimum capacity and higher priority.
- Reporting Queue – fair-share capacity for interactive or scheduled reports.
- ML Queue – controlled resource allocation and concurrency limits.
- Use queue-level limits to prevent one workload from consuming the entire cluster.

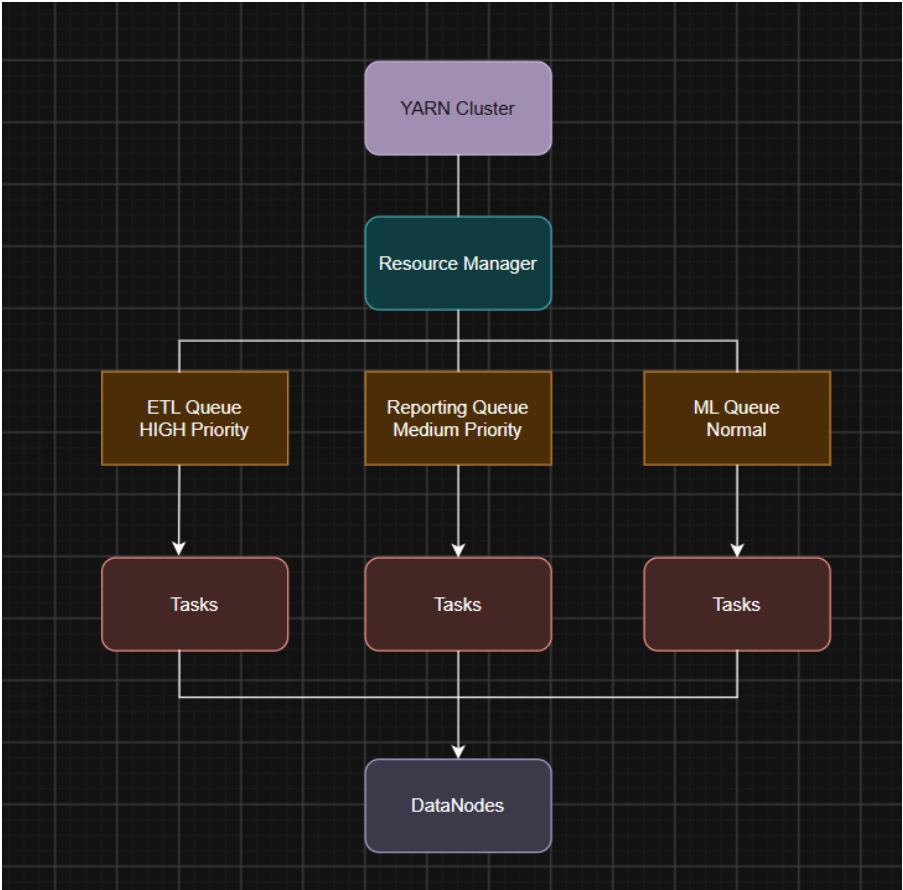
Node Failure Recovery

- YARN detects failed NodeManagers through heartbeat and health monitoring.
- Failed or incomplete containers are rescheduled on healthy nodes.
- MapReduce tasks can be re-executed because intermediate and source data remain available through HDFS.
- Use application retry policies and checkpointing where supported by the workload.
- Maintain sufficient spare capacity so recovery tasks can be scheduled after a failure.

Fairness and Priority Balance

- ETL receives higher priority but should not permanently starve reporting and ML workloads.
- Fair-share scheduling distributes unused capacity dynamically.
- Resource quotas and maximum application limits prevent resource monopolization.
- During peak ETL periods, other workloads continue to receive their minimum or fair share.





Question 4 – Constrained Enterprise Data Ingestion Architecture

Objective

Ingest data from SAN, NAS, and operational databases into a big data platform while maintaining reliable backup and minimizing duplicate data.

Key Constraints

- Multiple heterogeneous source systems must be integrated.
- Backup reliability must be maintained.
- Duplicate ingestion should be minimized.
- The architecture should scale as source data grows.

Proposed Architecture

SAN / NAS / Operational Databases → Ingestion Layer → Validation & Deduplication → HDFS Data Lake → Processing/ETL → Curated Data → Backup/Archive

Component Responsibilities

Component	Responsibility	Constraint Addressed
Ingestion Layer	Collect data from heterogeneous sources	Supports multiple source types.
Metadata/Schema Registry	Track source, schema, timestamps, and versions	Improves consistency and duplicate detection.
Validation Layer	Validate records and reject malformed data	Improves data reliability.
Deduplication	Use business keys, checksums, timestamps, or source IDs	Minimizes duplicate records.
HDFS	Store scalable raw and processed datasets	Provides distributed storage.
Backup/Archive	Maintain secondary copies according to policy	Supports recovery and reliability.
ETL/Processing	Transform and curate data	Creates analytics-ready datasets.

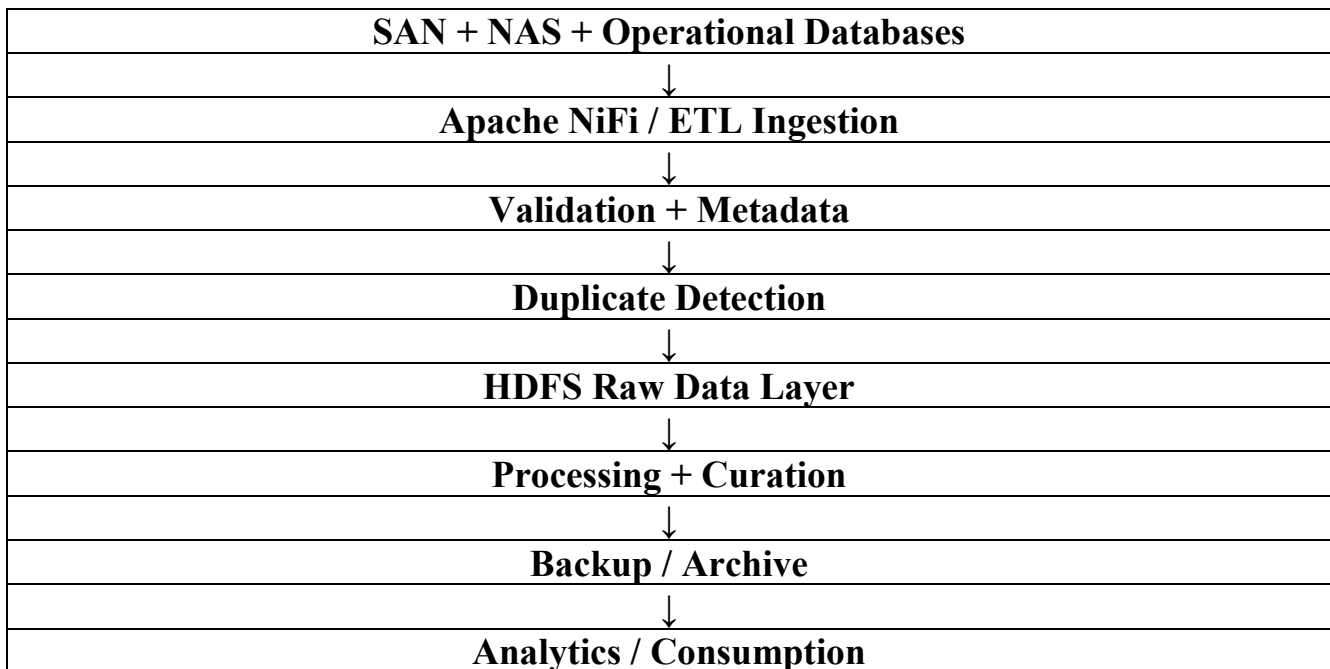
Deduplication Strategy

- Assign a unique source identifier to each source record or batch.

- Use file checksums for file-level duplicate detection.
- Use business keys and event timestamps for record-level deduplication.
- Maintain ingestion metadata including source, batch ID, ingestion time, and processing status.
- Make ingestion jobs idempotent so a retry does not create additional copies.

Backup Reliability

- Use HDFS replication within the permitted storage design.
- Maintain backup or archive copies according to recovery requirements.
- Test restoration periodically rather than assuming that backup creation alone guarantees recoverability.
- Separate operational source backups from analytics copies where appropriate.



Question 5 – End-to-End Hadoop Ecosystem Solution

Objective

Design a secure, highly available Hadoop ecosystem integrated with Apache NiFi or ETL pipelines for data ingestion and processing.

Key Constraints

- Secure data movement is required.
- High availability must be maintained.
- The solution must integrate with Apache NiFi or ETL pipelines.
- The Hadoop ecosystem must support scalable storage and processing.

End-to-End Architecture

Data Sources → Apache NiFi / ETL → Secure Transport → HDFS → YARN → MapReduce / Other Processing → Curated Data → Analytics / Reporting

Security Design

- Encrypt data in transit between ingestion components and Hadoop services.
- Use authentication and authorization mechanisms appropriate to the Hadoop deployment.
- Apply least-privilege access to datasets and services.
- Maintain audit logs for access and data movement.
- Protect sensitive data using appropriate encryption and access controls.

High Availability Design

- Use NameNode high availability with Active and Standby NameNodes.
- Distribute DataNodes across failure domains.
- Use YARN ResourceManager high availability where required.
- Configure retry and recovery mechanisms for failed tasks and containers.

- Use redundant ingestion paths or NiFi components where the deployment requires continuous ingestion.

Apache NiFi / ETL Integration

- NiFi can ingest data from multiple sources and route it to Hadoop storage.
- Use processors for transformation, filtering, routing, and validation before writing to HDFS.
- Track provenance and flow status for operational visibility.
- Use ETL pipelines for scheduled transformations and curated datasets.
- Connect downstream MapReduce or other Hadoop processing jobs to the curated HDFS data.

Constraint Satisfaction Summary

Constraint	Design Response
Secure data movement	Encrypted transport, authentication, authorization, and auditing.
High availability	NameNode HA, distributed DataNodes, ResourceManager HA, and task recovery.
NiFi/ETL integration	NiFi or ETL ingestion layer connected to HDFS and downstream processing.
Scalability	Horizontal DataNode expansion and distributed processing through YARN.
Operational reliability	Monitoring, retries, provenance, and controlled access.

Overall Design Summary

- HDFS provides scalable, distributed storage with replication and failure tolerance.
- MapReduce provides parallel batch processing through Mapper, Shuffle/Sort, and Reducer stages.
- YARN provides resource management, workload isolation, scheduling, and recovery.
- Apache NiFi or ETL pipelines provide controlled ingestion, transformation, routing, and provenance.
- Security, high availability, deduplication, and cost efficiency should be treated as cross-cutting architecture requirements.

- Every design decision should be explicitly linked to the constraints stated in the assessment questions.

Q5 End-to-End Hadoop Flowchart

